0.1 TL;DR

- **Whitepaper is the ground truth**; CIPs are improvements layered on top.
- **Audit reports are the source of truth** for what's missing.
- **Linear is the canonical place** for tickets and progress — one project per CIP and per whitepaper section.
- **A feature is Done when it's promoted to Mesa**, not when its PR merges to `main`.
- Tony's team produces **weekly progress reports**; Linear mirrors current implementation state.

0.2 1. Linear structure

0.2.1 Initiative

All protocol work lives under one initiative: **Validator**.

0.2.2 Projects

One project per spec unit, attached to the Validator initiative:

- **CIP projects** — one per CIP (CIP-1 through CIP-9, 12, 13, 15, 18, 20–26, 28–31).
- **Whitepaper-derived projects** — 6 projects covering the foundational pieces that aren't a CIP:
  - Consensus, P2P & Randomness (WP §6)
  - Accounts, Transactions & Mempool (WP §§1–2)
  - State Transition Function, System Actors & Precompiles (WP §§3, 9)
  - Python Actor VM (WP §3)
  - Data Availability, Blobs & Storage (WP §§4, 7)
  - Economics, Entitlements & Genesis (WP §§8, 13, 15)

0.2.3 Project statuses

Status	Meaning
Backlog	Not yet considered / not on the roadmap

Status	Meaning
Planned	On the roadmap; not started
Audited / Awaiting Implementation	Spec reviewed, gaps filed as issues, no implementation work has begun
In Progress	At least one working subsystem shipped or actively under development
Blocked	Waiting on a dependency in another project (resume when the dep lands)
Completed	Promoted to Mesa and considered done
Canceled	Won't pursue; superseded; or merged into another project

---

0.3 2. Issue workflow

Each project carries issues for individual gaps. Issues move through these states:

State	When
Backlog	Filed but not picked up
Todo	Ready to be worked on next
In Progress	Engineer is actively coding
In Review	PR open against `main`
Testing	Merged to `main`; deployed to Canyon devnet for testing
Done	Promoted from Canyon to Mesa and verified there
Canceled	Won't do; obsoleted by a spec change; or superseded

The state advances **only** when the work reaches the next stage on a real devnet — not when the engineer thinks it's done.

0.3.1 Issue labels

- Severity labels: `severity:high`, `severity:medium`, `severity:low`.
- Type labels: `Feature`, `Improvement`, `Bug`, `Documentation`.

0.3.2 Sub-issues

For non-trivial CIP tasks, use sub-issues under a parent ticket to break a feature into reviewable units. The parent reflects overall progress; sub-issues carry their own state. This keeps the project view scannable without losing per-task granularity.

0.4 3. Whitepaper / CIP relationship

The whitepaper and the CIPs are not parallel sources — they nest:

- **Whitepaper** is the **canonical, self-contained** description of how the protocol works. A reader should be able to understand the system from the whitepaper alone.
- **CIPs** are **improvements layered on top** of the whitepaper. Each CIP describes a feature, a refinement, or a parameter set, with the level of detail an implementer needs.
- Whitepaper sections that currently include `see CIP-X` cross-references will be **rewritten to inline a summary** of the CIP's intent. **The CIP itself remains the authoritative detailed spec** — only summaries get migrated. CIPs are not deprecated by this process.

When a CIP and the whitepaper disagree on a parameter, an opcode, or a default, **the whitepaper wins** unless the CIP explicitly amends it (and the amendment is signed off). Discrepancies surfaced during audits become issues against either the whitepaper project or the CIP project, depending on which side is wrong.

0.5 4. Audit reports

0.5.1 The baseline

The **May 2026 audit pass** is the baseline — every CIP and whitepaper section that was triaged has a verdict + a list of filed gaps. The verdict lives in the project's content (look for the `<!-- audit:cip-audit-2026-05 -->` block).

0.5.2 Weekly progress reports

Tony's team produces a **weekly completion report** that records, per CIP:

- Overall completion percentage
- What was implemented in the last week
- What's still missing (with sub-item granularity)

These reports are the working document; Linear mirrors them. When the report says an item is done, the corresponding Linear issue should move to **Done**. When the report surfaces a new gap, a new issue should appear.

0.5.3 Sync (in progress)

A script is being built to automate the audit-report → Linear sync. Until it lands, the team that owns a CIP project is responsible for keeping its issue list in sync with the weekly report.

0.6 5. Promotion flow

Code moves through three environments before a feature is considered done:

```
write code
    |
    v
feature branch  -->  PR  -->  main
                                |
                                v
                          Canyon (devnet)   <-- deployed automatically
                                |
                                v   (tag + promote when stable)
                          Mesa (production) <-- done
```

Done = promoted to Mesa. A PR merging to `main` only moves the issue to **Testing on Canyon**. The issue closes when the feature is verified in Mesa.

0.6.1 Weekly merge cadence

`devnet` is rebased on `main` weekly, and a PR merging `devnet` back into `main` is opened at the same cadence. This keeps the two branches from diverging and gives operators a predictable cutover point.

0.7 6. Where to look

Question	Where
"What's the state of CIP-X?"	The CIP-X project's verdict block + active issues
"What's high-priority across everything?"	Filter Linear issues by `severity:high`
"What was on the May 2026 baseline?"	Filter by issue label `cip-audit-2026-05`
"Which projects have been audited at all?"	Filter projects by `audited-26-05`
"Which projects are not yet started?"	Project status = **Audited / Awaiting Implementation** or **Backlog**
"Which projects are blocked on a dep?"	Project status = **Blocked**
"What landed last week?"	Tony's latest weekly progress report

---

0.8 7. Conventions

- Don't delete tickets that turn out to be wrong; **cancel** them with a comment pointing to the replacement. Keeps the audit trail intact.
- Don't change a project's name once it's filed; spec rewrites get a project-update post in the activity feed and an updated audit-verdict block.
- One PR per logical change. Don't bundle a CIP-2 fix and a CIP-9 fix into one PR.
- A new system actor opcode requires updating CIP-13 §1's master allocation table in the same PR that introduces the opcode.